WEB APPLICATION BASICS

TryHackMe Learning & Practical Report

Prepared by: Sofia
Focus: Web Application Security / WAPT Foundations
Platform: TryHackMe
Room: Web Application Basics
IqraSec Academy - WAPT Internship - Phase 1

1. Executive Summary

Completed TryHackMe Web Application Basics room as first step WAPT training. The room explains how browser & web server talk to each other, how a URL is built, how HTTP requests and responses look and which security headers help protect an application. In the practical part I used an API with GET, POST and DELETE requests, which showed me how one endpoint can read change or remove data depending on the method.

2. Lab Information

Platform	TryHackMe
Room / Lab	Web Application Basics
Primary Domain	Web Application Security
Purpose	Foundational WAPT practice
Completion	Completed - all room tasks finished
Link	https://tryhackme.com/room/webapplicationbasics

3. Learning Objectives

- Identify the main parts of a web application (browser, web server, WAF).
- Understand the parts of a URL and why query strings matter for security.
- Understand HTTP request and response messages.
- Know what the common HTTP methods are used for.
- Understand status codes and security headers (CSP, HSTS, nosniff).
- Practise basic API requests using GET, POST and DELETE.

4. Web Application Architecture

A web application has a client (the browser), a web server that hosts and delivers the application, and often a Web Application Firewall (WAF) that sits in front and filters malicious traffic. For a tester every layer is a different place where something can go wrong.

5. URL Structure and Security Relevance

A URL has a scheme (HTTP or HTTPS), a host/domain, a port (80 for HTTP and 443 for HTTPS by default), a path, a query string after ? sign and a fragment after #. The query string is controlled by the user so it is the first place attackers try injections.

Typosquatting means registering a domain that looks almost like a real one so that users visit it by mistake. HTTPS gives encrypted communication between browser and server.

6. HTTP Messages, Requests and Responses

A request has a request line (method, path, version), headers, an empty line and an optional body. A response has a status line, headers, an empty line and the body. Important request headers are Host, User-Agent and Content-Type (for example application/x-www-form-urlencoded for form data). The server answers with a status code: 2xx success, 3xx redirect, 4xx client error such as 404 Not Found and 5xx server error.

7. Security Headers

Content-Security-Policy (CSP) - controls where scripts and other resources may load from, for example with script-src.

X-Content-Type-Options: nosniff - stops the browser from guessing a different MIME type.

8. Practical Task - API Requests

The last task of the room gave an API that I could call from the browser. I made three requests:

GET /api/users - returned the list of users and gave the first flag.

POST /api/user/1 - changed the country of the user Bob from UK to US.
DELETE /api/user/1 - removed the user; the room showed the flag THM{YOU_HAVE_JUST_DELETED_A_USER}.
The same URL did different things only because the method was different. This is why the server must check authorisation for every method and not only for GET.

9. Evidence

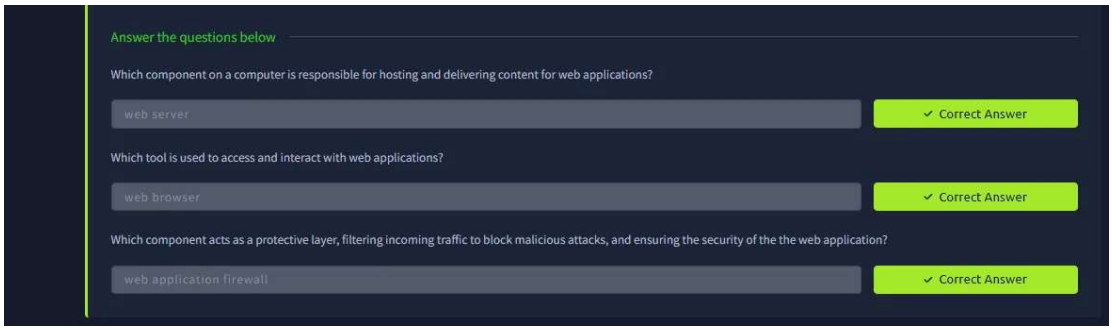


Fig 1 - Web application components (web server, web browser, WAF).

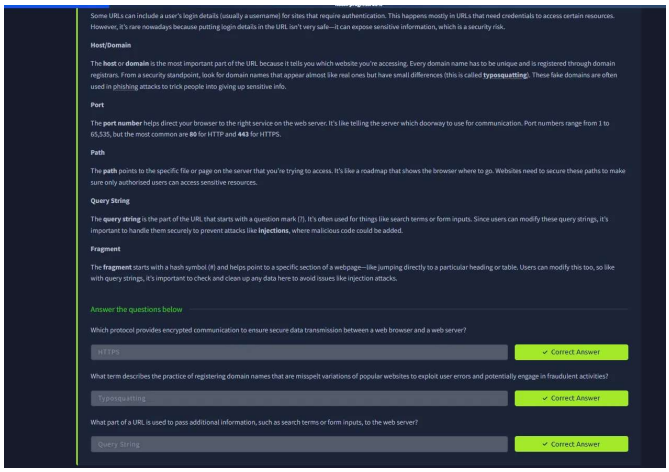


Fig 2 - URL parts, HTTPS, typosquatting and query string.

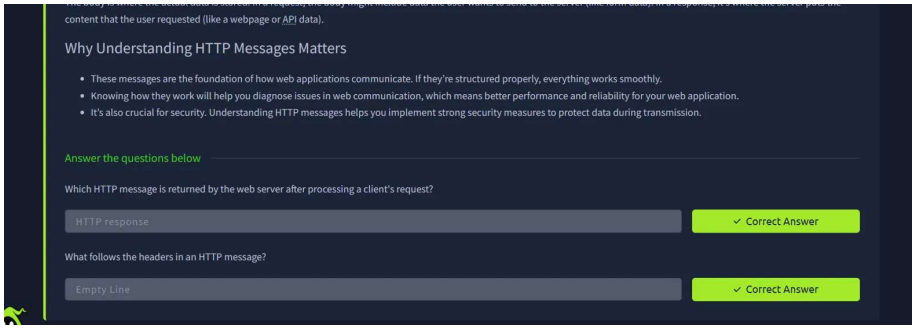


Fig 3 - HTTP response and the empty line after the headers.

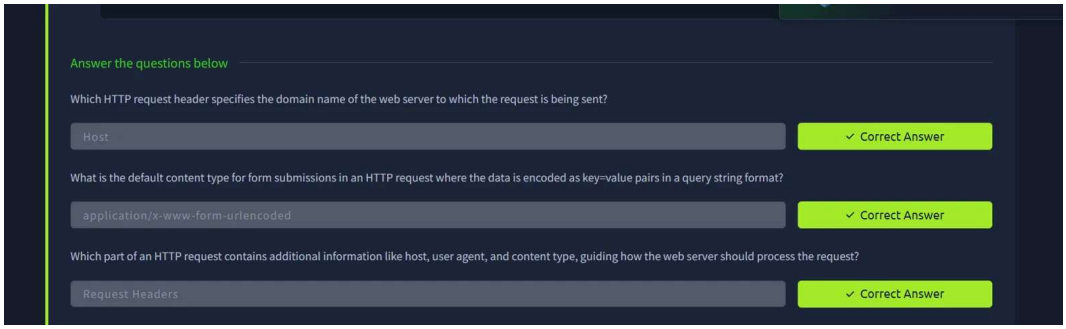


Fig 4 - Host header, form content type and request headers.

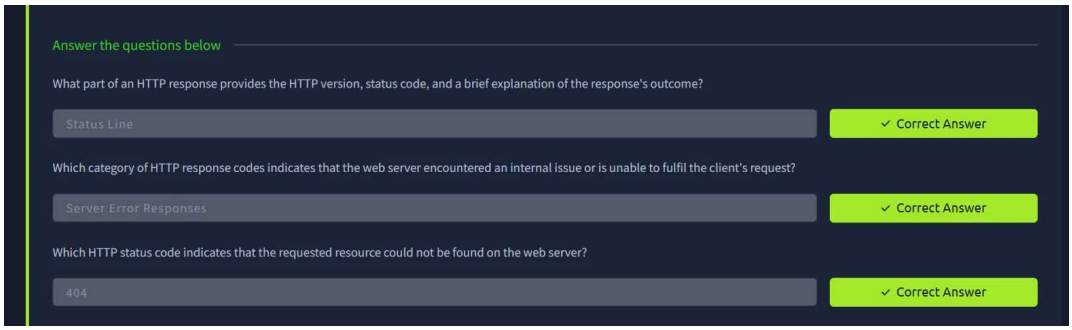


Fig 5 - Status line, server error responses and 404.

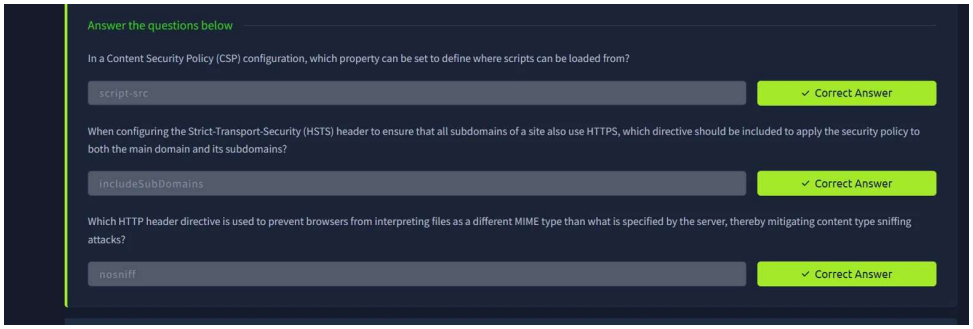


Fig 6 - CSP script-src, HSTS includeSubDomains and nosniff.

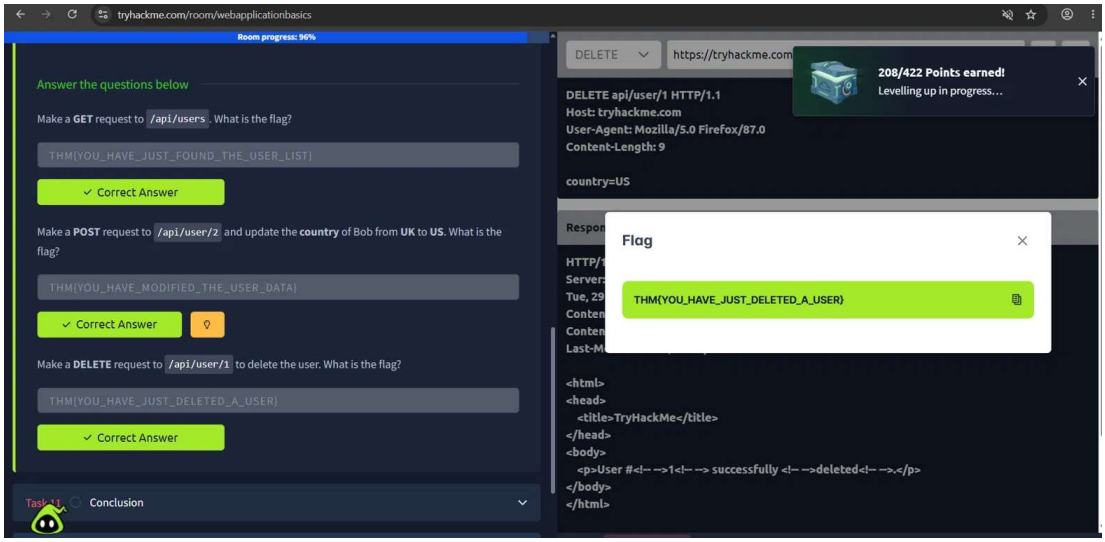


Fig 7 - Practical task: DELETE request and the flag.

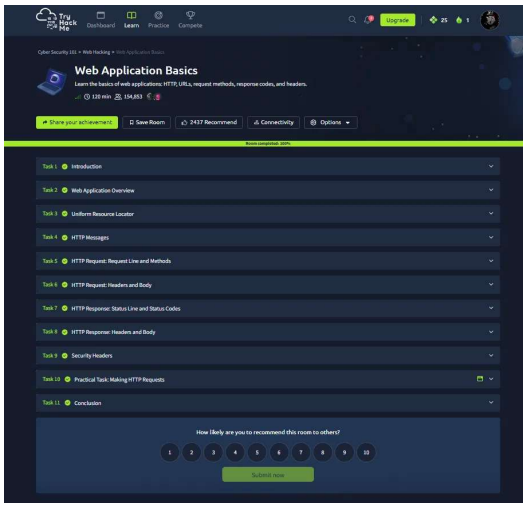


Fig 8 - Room tasks completed.

10. Key Security Takeaways

Every web attack is a changed HTTP request, so I must understand normal requests first.
HTTP methods show the intended action, but the server has to enforce authorisation itself.
URL parameters and request bodies are user input and must never be trusted.
Status codes and response differences are useful signals while testing.
Security headers give extra protection and should be checked in every assessment.

11. Skills Practiced

Area	**Practical understanding gained**
Web basics	Client, server and WAF roles
URLs	Scheme, host, port, path, query string
HTTP	Request and response structure
Methods	GET, POST and DELETE
Headers	Host, Content-Type, CSP, HSTS, nosniff
Status codes	Reading 2xx, 4xx and 5xx responses
APIs	Basic retrieve, modify and delete requests

12. What I Would Explore Next

Next I want to use Burp Suite to intercept and change these requests myself, and then move to authentication, access control, SQL injection and XSS.

13. Conclusion

This room changed how I look at a website: instead of only a page, I now see requests, responses, parameters and headers. The API practical showed me how easy it is to change or delete data if the server does not check permissions. This is the base I will use for the rest of Phase 1.
Completion status: Completed - all room tasks finished.

14. References

TryHackMe: https://tryhackme.com/room/webapplicationbasics